Design and Development of AMBA AHB-Lite Verification IP

Using SystemVerilog OOP and Assertion-Based Verification

Presented by:
Diyaa Shashidharan - NNM23VL024
Krishna M Nair - NNM23VL034
Leron Lawrence Crasta - NNM23VL035
Navaneeth CP – NNM23VL038

Introduction

•Modern SoCs use standard buses for communication between processor, memory, and peripherals

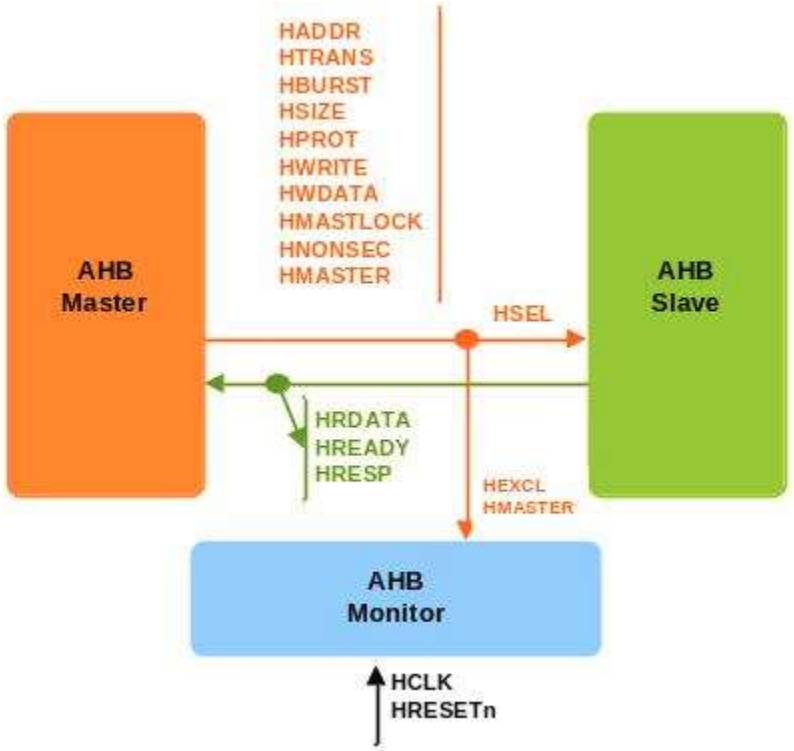
•AMBA AHB-Lite is a high-performance bus protocol used for this communication

•It supports pipelined transfers for efficient data throughput

•However, verifying such protocols is complex due to multiple signal interactions

Problem Statement:
Traditional verification methods are not reusable, scalable, or efficient for complex SoC protocols

Solution:
Develop a reusable Verification IP (VIP) for AMBA AHB-Lite using System Verilog



Objectives

- Design a simple AHB-Lite slave/memory as DUT
- Build a class-based SystemVerilog verification environment
- Apply OOP concepts like encapsulation and inheritance
- Verify protocol behavior using SystemVerilog Assertions (SVA)
- Implement functional coverage for different transaction scenarios
- Validate correct read, write, and burst operations

AHB-Lite Basics

•AHB-Lite is a single-master bus protocol used in SoC communication

Key Signals:

•HADDR → Address bus

•HWDATA → Write data bus

•HRDATA → Read data bus

•HTRANS → Transfer type

•HREADY → Indicates slave ready status

•HRESP → Response signal (OKAY/ERROR)

Operation:

•Address Phase → Data Phase (pipelined execution)

Transfer Types:

•IDLE, NONSEQ, SEQ

Verification Architecture

•Developed a modular, reusable SystemVerilog testbench

Components:

•Generator → Generates random transactions

•Driver → Drives stimulus to DUT

•DUT → AHB-Lite slave/memory model

•Monitor → Observes bus activity

•Scoreboard → Compares expected and actual results

This forms a complete Verification IP environment for AHB-Lite

OOP Testbench Components

- **Transaction Class:** Defines address, data, control signals
- **Generator:** Creates constrained-random transactions
- **Driver:** Converts transactions into pin-level signals
- **Monitor:** Captures DUT responses and reconstructs transactions
- **Scoreboard:** Validates DUT output against expected results
- **Environment:** Connects all components and controls simulation flow

This OOP-based structure improves reusability and scalability

Assertion-Based Verification

•SystemVerilog Assertions (SVA) are used for protocol checking

•Helps detect violations automatically during simulation

Key Checks:

•Address must remain stable during transfer

•Proper handshake between HTRANS and HREADY

•Valid state transitions in transfer cycle

•Correct response timing using HRESP

Functional Coverage

•Ensures all important scenarios are verified

Coverage Points:

•Read and Write operations

•Transfer types: IDLE, NONSEQ, SEQ

•Burst transactions

•Response types: OKAY and ERROR

•Coverage helps measure completeness of verification

•Target coverage achieved: ~90% or higher

Simulation Results

•Simulation performed using System Verilog testbench

Observations:

•Correct separation of address and data phases

•Proper execution of read/write operations

•HREADY correctly handles wait states

•No protocol violations observed

•Assertions passed successfully

Conclusion

•Successfully developed reusable AHB-Lite Verification IP

•Verified protocol using OOP-based testbench, assertions, and coverage

•Ensured correct functional behavior of DUT

Future Scope:

•Extend to full AMBA AHB with multi-master support

•Upgrade environment to UVM-based verification

References: